
Tecnologie del Linguaggio Naturale

Parte Prima

Lezione n. 05

> Formal and Computational Semantics

6 Marzo, 2026

slides strongly based on Bill MacCartney <http://nlp.stanford.edu/~wcmac>

Outline

- SHRDLU & Chat-80
- Computational Semantics fundamentals
- λ abstraction
- CS per:
 - >Articoli indeterminativi ->Nomi propri ->Verbi transitivi
 - ~~->Cordinazione dei nomi ->Quantificatori universali~~
- NLTK

Outline

- SHRDLU & Chat-80
- Computational Semantics fundamentals
- λ abstraction
- CS per:
 - >Articoli indeterminativi ->Nomi propri ->Verbi transitivi
 - ~~->Cordinazione dei nomi ->Quantificatori universali~~
- NLTK

Si possono costruire le grammatiche a mano?

- Per l'italiano: L. Lesmo (1985 -> 2014), Common Lisp
- Per l'inglese:
 - SHRDLU (Winograd 1972)
 - <https://www.youtube.com/watch?v=QAJz4YKUwqw>
 - <http://hci.stanford.edu/winograd/shrdlu/>
 - CHAT-80: 1979-82 F. Pereira & D. Warren, Prolog
 - Linguaggio naturale per un DB sulla geografia
 - Where is Rome?
 - Which country bordering the Mediterranean borders a country that is bordered by a country whose population exceeds the population of India?

<https://nli-gems.org/all-systems#chat-80>

Prolog :)

```
/* Sentences */
sentence(S) --> declarative(S), terminator(.) .
sentence(S) --> wh_question(S), terminator(?) .
sentence(S) --> yn_question(S), terminator(?) .
sentence(S) --> imperative(S), terminator(!) .

/* Noun Phrase */
np(np(Agmt, Pronoun, []), Agmt, NPCase, def, _, Set, Nil) -->
    {is_pp(Set)},
    pers_pron(Pronoun, Agmt, Case),
    {empty(Nil), role(Case, decl, NPCase)} .

/* Prepositional Phrase */
pp(pp(Prep, Arg), Case, Set, Mask) -->
    prep(Prep),
    {prep_case(NPCase)},
    np(Arg, _, NPCase, _, Case, Set, Mask) .
```

Outline

- SHRDLU & Chat-80
- **Computational Semantics fundamentals**
- λ abstraction
- CS per:
 - >Articoli indeterminativi ->Nomi propri ->Verbi transitivi
 - ~~->Cordinazione dei nomi ->Quantificatori universali~~
- NLTK

Rappresentare il significato

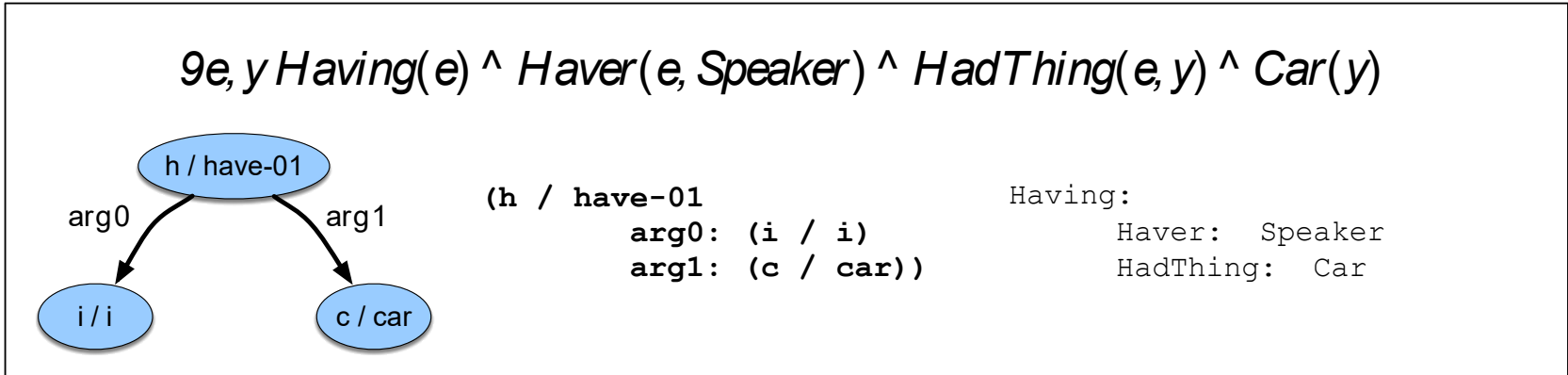


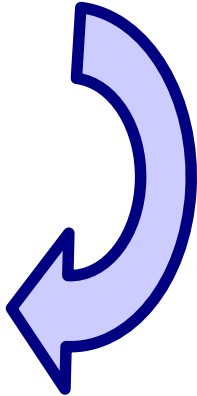
Figure F.1 A list of symbols, two directed graphs, and a record structure: a sampler of meaning representations for *I have a car*.

Importantly, these representations can be viewed from at least two distinct perspectives in all of these approaches: as **representations of the meaning of the particular linguistic input *I have a car***, and as **representations of the state of affairs in some world**. It is this dual perspective that allows these representations to be used to link linguistic inputs to the world and to our knowledge of it.

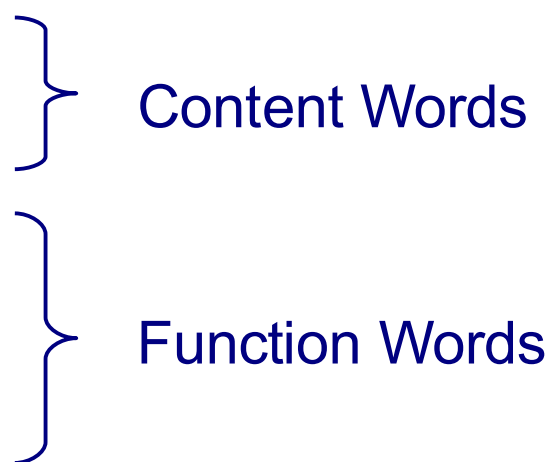
Rappresentare il significato

- Quale forma per il significato?
 - DB tables, SQL, description logics, modal logics, AMR, frames, ...
- Blackburn & Bos -> first-order logic (FoL)
 - Paolo corre -> run(Paolo)
 - Paolo ama Francesca -> love(Paolo, Francesca)
 - Tutti gli uomini amano Francesca -> $\forall x (\text{man}(x) \rightarrow \text{love}(x, \text{Francesca}))$
- Il problema nasce per le “frasi incomplete”
 - ama -> love(x,y)
- **Lambda-FoL termini per parole e sintagmi -> FoL per le frasi**

Computational Semantics

- Frase
 - Paolo corre. Tutti quelli che corrono vivono meglio.
 - Syntactic analysis
 - (S (NP Paolo) (VP corre))
 - Semantic interpretation/analysis
 - `run(paolo)`
 - Inference
 - $\forall x.\text{run}(x) \rightarrow \text{goodLife}(x)$
 - $\text{run}(\text{paolo}) \rightarrow \text{goodLife}(\text{paolo})$
- 
- Computational
Semantics**

FoL syntax in una slide

- FOL simboli
 - Costanti: **john, mary**
 - Predicati & relazioni: **man, walks, loves**
 - Variabili: **x, y**
 - Connettivi Logici: $\wedge \vee \neg \rightarrow$
 - Quantificatori: $\forall \exists$
 - Altri simboli ausiliari: $() ,$
 - FOL formule
 - Formule atomiche: loves(paolo,francesca)
 - Formule composte: man(paolo) $\wedge$ loves(paolo,francesca)
 - Formule quantificate: $\exists x$ (man(x))
- 
- Content Words
- Function Words

FoL e ristoranti

Formula $\rightarrow$ *AtomicFormula*
| *Formula* *Connective* *Formula*
| *Quantifier* *Variable*, ... *Formula*
| $\neg$ *Formula*
| (*Formula*)
AtomicFormula $\rightarrow$ *Predicate*(*Term*, ...)
Term $\rightarrow$ *Function*(*Term*, ...)
| *Constant*
| *Variable*
Connective $\rightarrow$ $\wedge$ | $\vee$ | $\Rightarrow$
Quantifier $\rightarrow$ $\forall$ | $\exists$
Constant $\rightarrow$ *A* | *VegetarianFood* | *Maharani* ...
Variable $\rightarrow$ *x* | *y* | ...
Predicate $\rightarrow$ *Serves* | *Near* | ...
Function $\rightarrow$ *LocationOf* | *CuisineOf* | ...

Il mondo dei ristoranti

Domain

Matthew, Franco, Katie and Caroline

Frasca, Med, Rio

Italian, Mexican, Eclectic

$\mathcal{D} = \{a, b, c, d, e, f, g, h, i, j\}$

a, b, c, d

e, f, g

h, i, j

Properties

Noisy

Frasca, Med, and Rio are noisy

$Noisy = \{e, f, g\}$

Relations

Likes

Matthew likes the Med

Katie likes the Med and Rio

Franco likes Frasca

Caroline likes the Med and Rio

$Likes = \{\langle a, f \rangle, \langle c, f \rangle, \langle c, g \rangle, \langle b, e \rangle, \langle d, f \rangle, \langle d, g \rangle\}$

Serves

Med serves eclectic

Rio serves Mexican

Frasca serves Italian

$Serves = \{\langle e, j \rangle, \langle f, i \rangle, \langle e, h \rangle\}$

Un modello per i ristoranti è ...

Abbiamo bisogno di mappare gli elementi di significato (formule FoL) al mondo

- FoL Termini $\rightarrow$ elementi del dominio
 - Med $\rightarrow f$
- FoL formule atomiche $\rightarrow$ insiemi sui domini di elementi o insiemi di tuple
- $\text{noisy}(\text{Med})$ è vera se f è nell'insieme degli elementi della relazione *noisy*
- $\text{near}(\text{Med}, \text{Rio})$ è vera se la tupla $\langle f, g \rangle$ è nell'insieme delle tuple che corrispondono a “*near*” nell'interpretazione

Un modello per i ristoranti è ...

- Per formule più grandi (1. connettivi) usiamo le tavole di verità

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$
<i>False</i>	<i>False</i>	<i>True</i>	<i>False</i>	<i>False</i>	<i>True</i>
<i>False</i>	<i>True</i>	<i>True</i>	<i>False</i>	<i>True</i>	<i>True</i>
<i>True</i>	<i>False</i>	<i>False</i>	<i>False</i>	<i>True</i>	<i>False</i>
<i>True</i>	<i>True</i>	<i>False</i>	<i>True</i>	<i>True</i>	<i>True</i>

- Per formule più grandi (2. quantificatori) usiamo la semantica di *per ogni* e di *esiste* ...

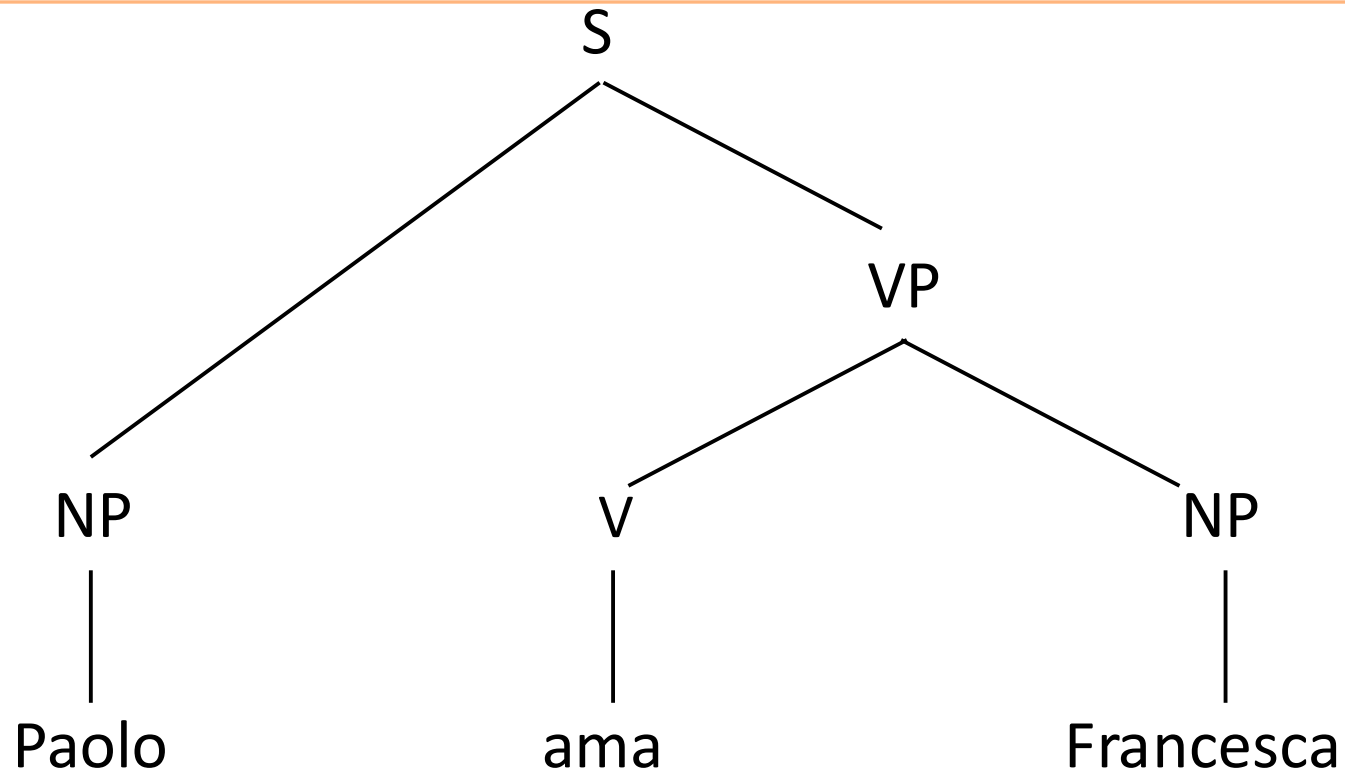
CS Fundamental Algorithm

1. Parsificare la frase per ottenere l'albero sintattico (costituenti)
2. Cercare la semantica di ogni parola nel lessico
3. Costruire la semantica per ogni sintagma
 - Bottom-up
 - Syntax-driven: “rule-to-rule translation”

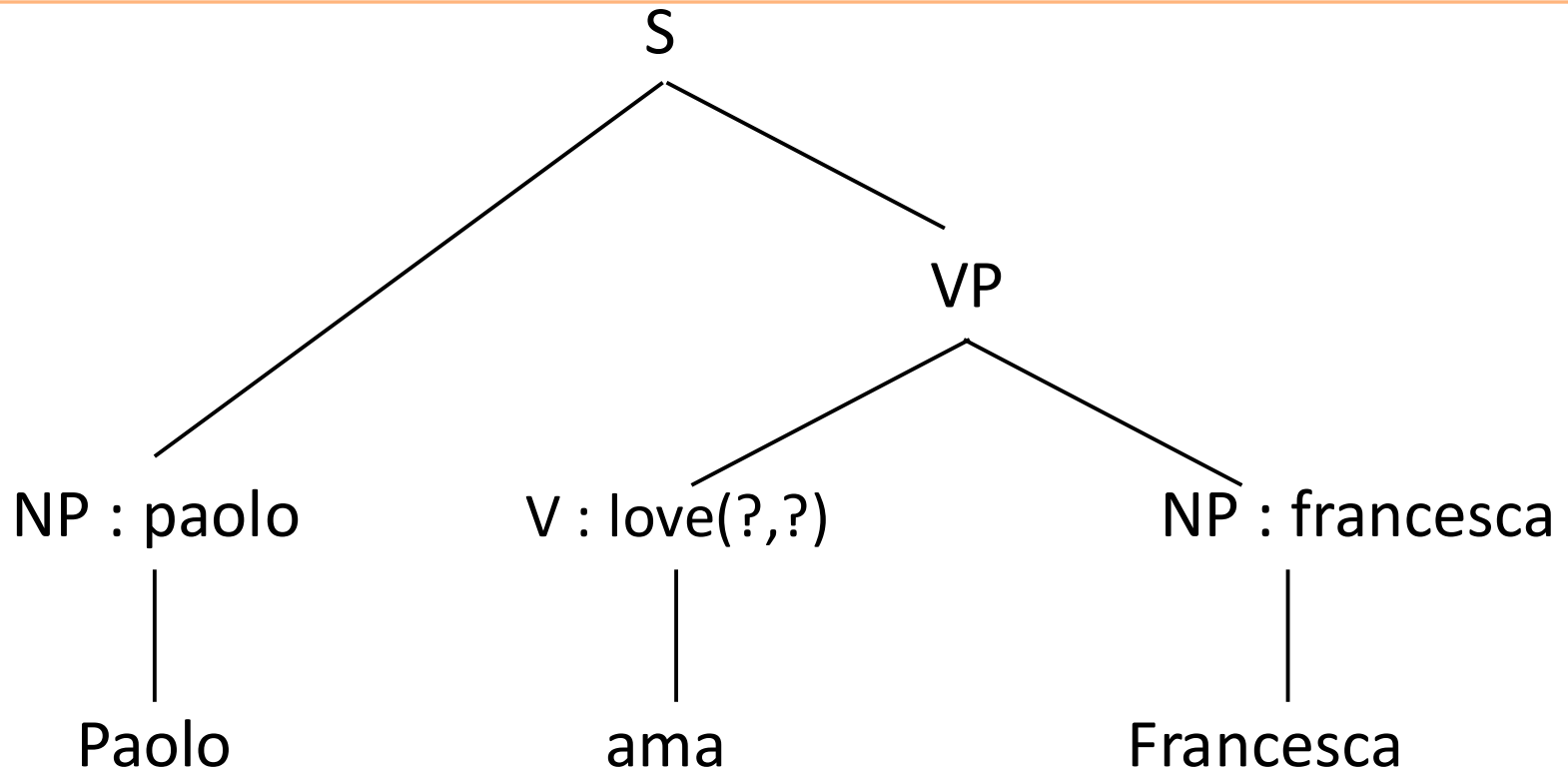
Principio di composizionalità di Frege

Il significato del tutto è determinato dal significato delle parti e dalla maniera in cui sono combinate

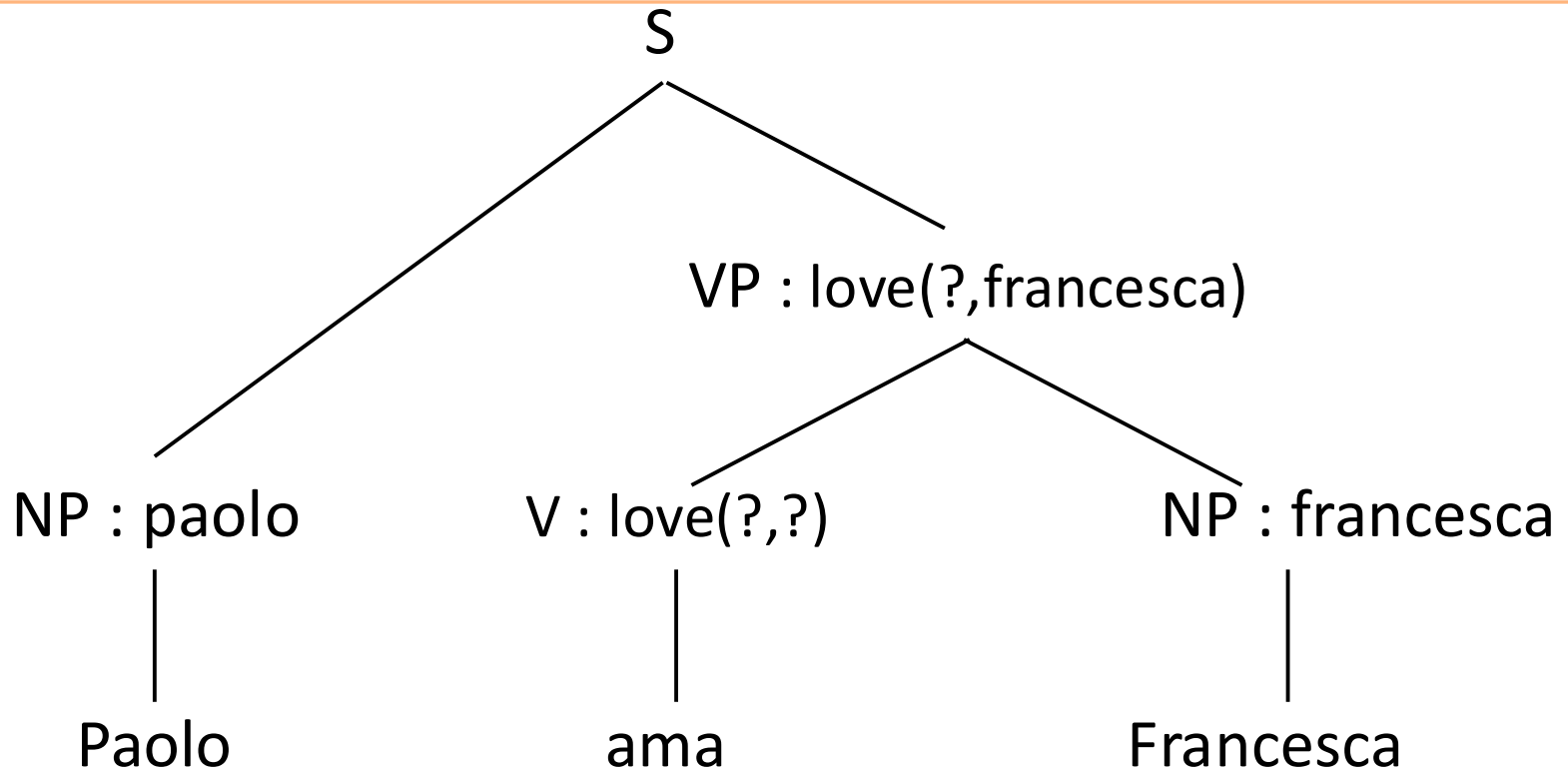
Esempio: analisi sintattica



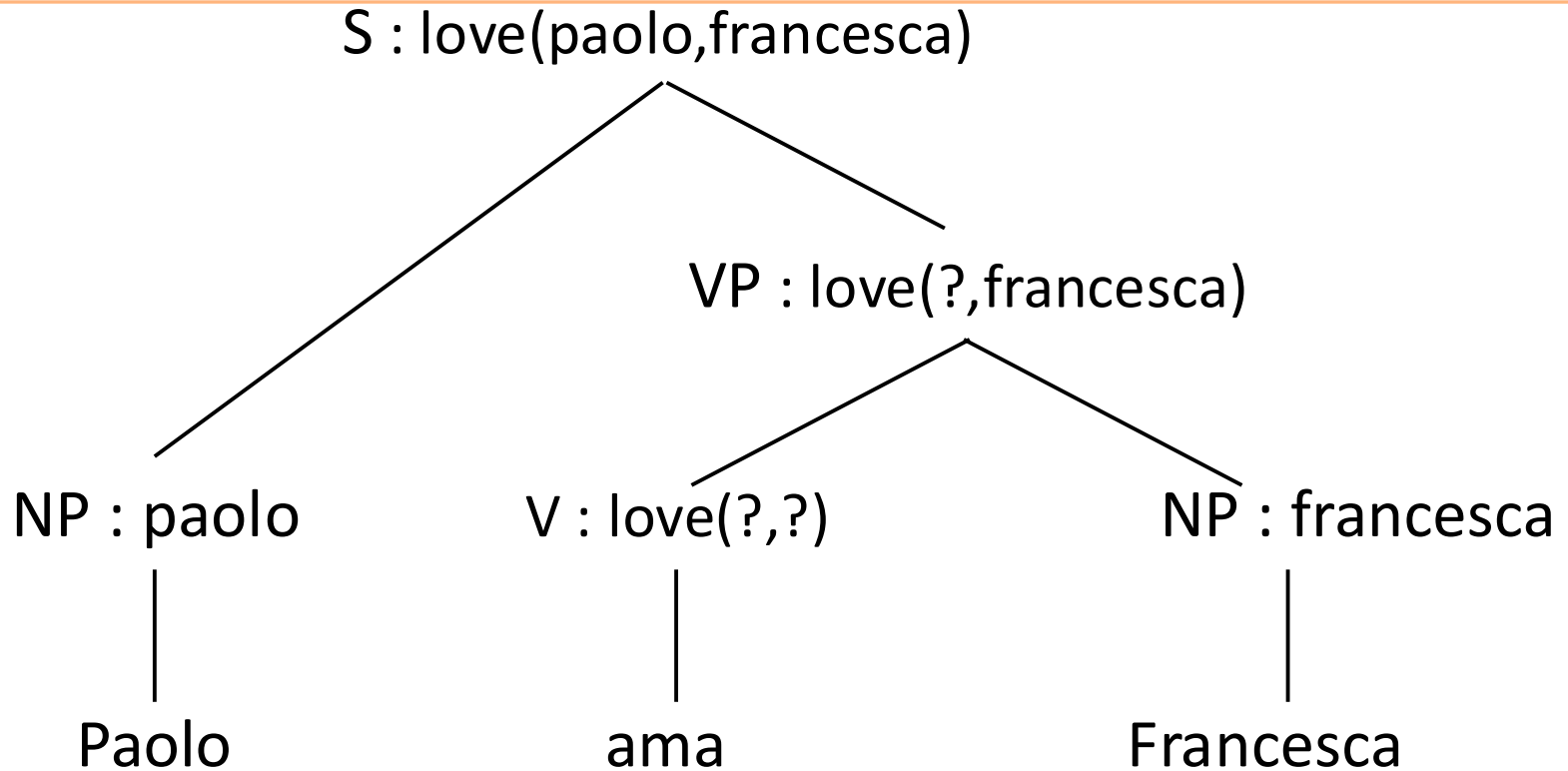
Esempio: lessico semantico



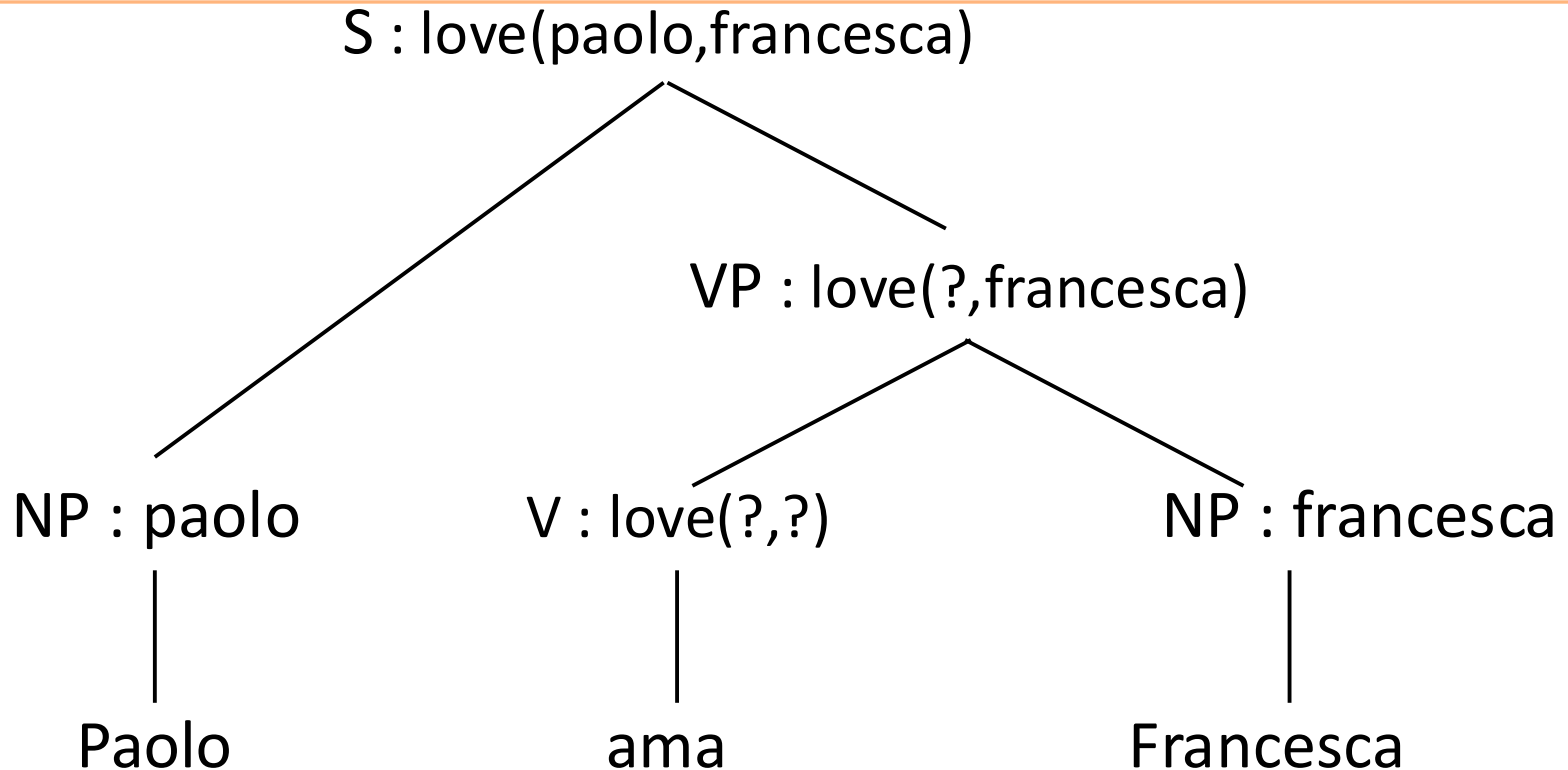
Esempio: composizione semantica



Esempio: composizione semantica



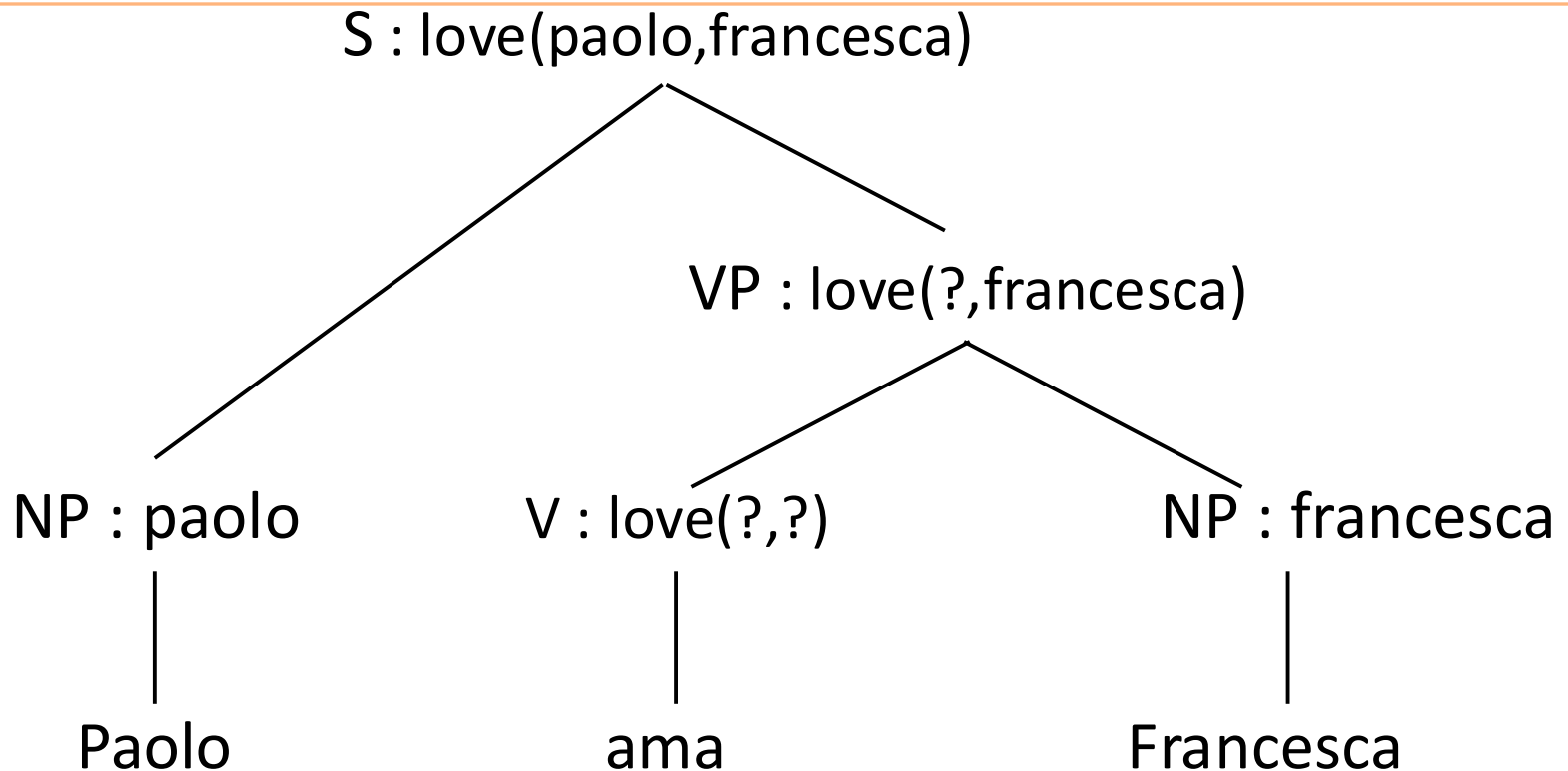
Composizionalità



Il significato della frase è costruito:

- dal significato delle parole -> lessico
- risalendo le costruzioni sintattiche -> regole semantiche

Sistematicità

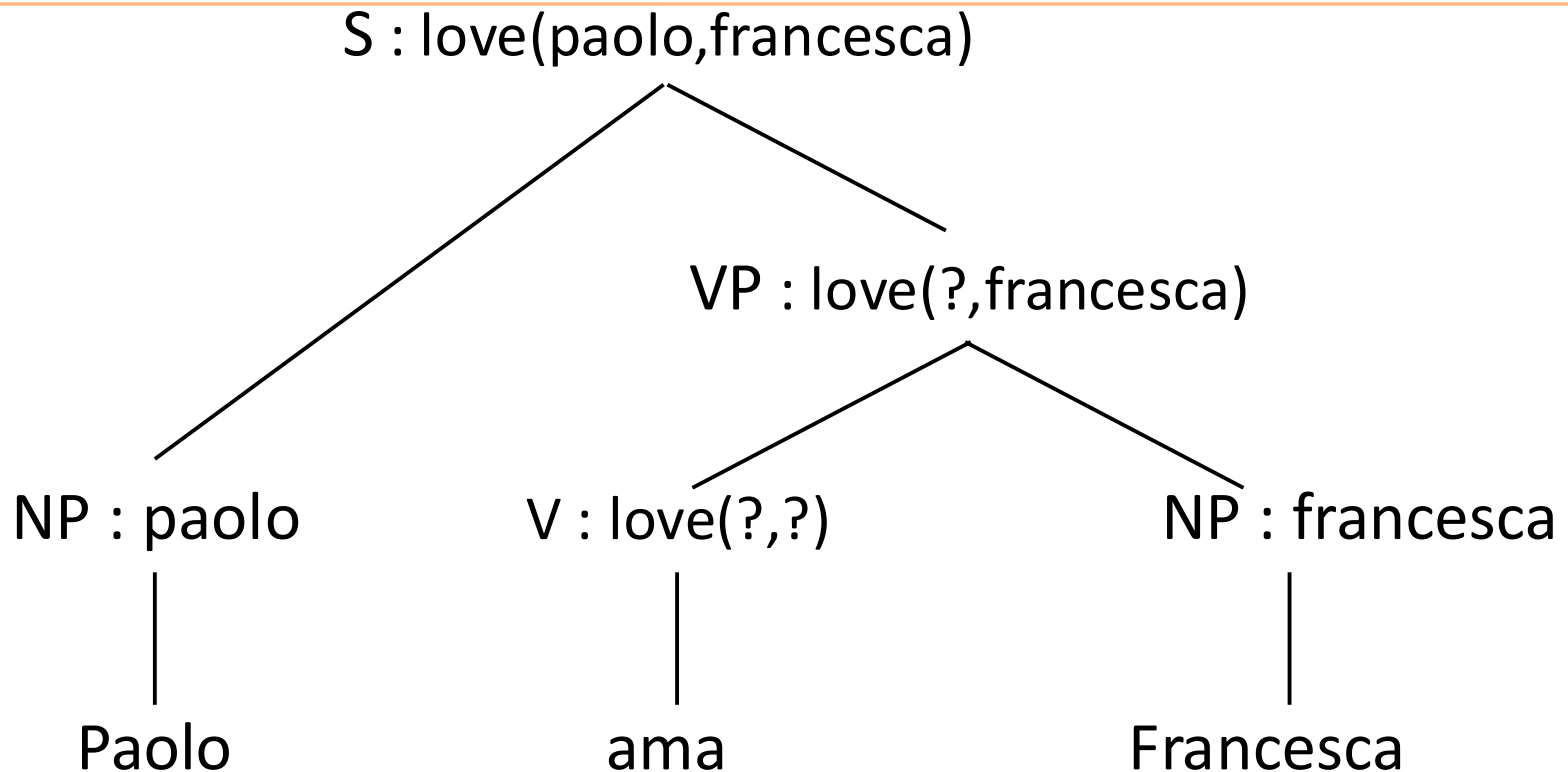


1. Come costruiamo il significato del VP?

love(?,francesca) o love(francesca,?)

2. Come specificare in che maniera combinare i pezzi?

Sistematicità



3. Come rappresentare pezzi di formule?

$\text{love}(?,\text{francesca}) \rightarrow$ non FoL, $\text{love}(x,\text{francesca}) \rightarrow$ variabile libera?

4. Vorremmo: *Tutti amano Francesca* $\rightarrow \forall x (\text{love}(x,\text{francesca}))$

Outline

- SHRDLU & Chat-80
- Computational Semantics fundamentals
- **λ abstraction**
- CS per:
 - >Articoli indeterminativi ->Nomi propri ->Verbi transitivi
 - ~~->Cordinazione dei nomi ->Quantificatori universali~~
- NLTK

Lambda abstraction

- Nuovo operatore λ per legare (bind) le variabili libere
 - $\lambda x.\text{love}(x, \text{francesca}) \rightarrow \text{amare Francesca}$
 - Il nuovo simbolo meta-logico λ segna l'informazione mancante nella meta-formula λ -FoL. $\rightarrow$ **astrarre su x**
 - Programmazione:
 - Common Lisp: `((lambda (x) (+ 1 x)) 7)`
 - [McCarthy58-Russel60]
 - Python: `lambda x: x % 2 == 0`
 - Java 8:
- <http://www.oracle.com/webfolder/technetwork/tutorials/obe/java/Lambda-QuickStart/index.html>
- Come combinare λ -FoL e FoL?

Function application

- $(\lambda x. \text{love}(x, \text{mary})) @ \text{john}$
- $(\lambda x. \text{love}(x, \text{mary}))(\text{john})$
- La FA attivà un'operazione elementare chiamata
- **Beta reduction** -> ...

Beta reduction

$(\lambda x. \text{love}(x, \text{mary})) (\text{john})$

1. Elimina il λ

$(\text{love}(x, \text{mary})) (\text{john})$

2. Rimuovi l'argomento

$\text{love}(x, \text{mary})$

3. Rimpiazza le occorrenze della variabile legata dal λ con l'argomento in tutta la formula

$\text{love}(\text{john}, \text{mary})$

Esempio: composizione semantica usando λ

$S : \lambda x.\text{love}(x,\text{francesca})(\text{paolo})$
 $= \text{love}(\text{paolo},\text{francesca})$

$VP : \lambda y.\lambda x.\text{love}(x,y)(\text{francesca})$
 $= \lambda x.\text{love}(x,\text{francesca})$

NP : paolo

Paolo

V : $\lambda y.\lambda x.\text{love}(x,y)$

ama

NP : francesca

Francesca

Una semplicissima grammatica semantica (1)

Lessico

NP : paolo -> Paolo

NP : francesca -> Francesca

V : $\lambda y. \lambda x. \text{love}(x, y)$ -> ama

Regole di composizione

VP : $f(a)$ -> V : f NP : a

S : $f(a)$ -> NP : a VP : f

Una semplicissima grammatica semantica (1)

Lessico

NP : paolo -> Paolo

NP : francesca -> Francesca

V : $\lambda y. \lambda x. \text{love}(x, y)$ -> ama

Regole di composizione

VP : $f(a)$ -> V : f NP : a

S : $f(a)$ -> NP : a VP : f

Augmented CFG == Grammatiche ad attributi

YACC `expr:expr '*' expr { $$=$1*$3};`

Semantica di Montague

Richard Montague (1930-71)

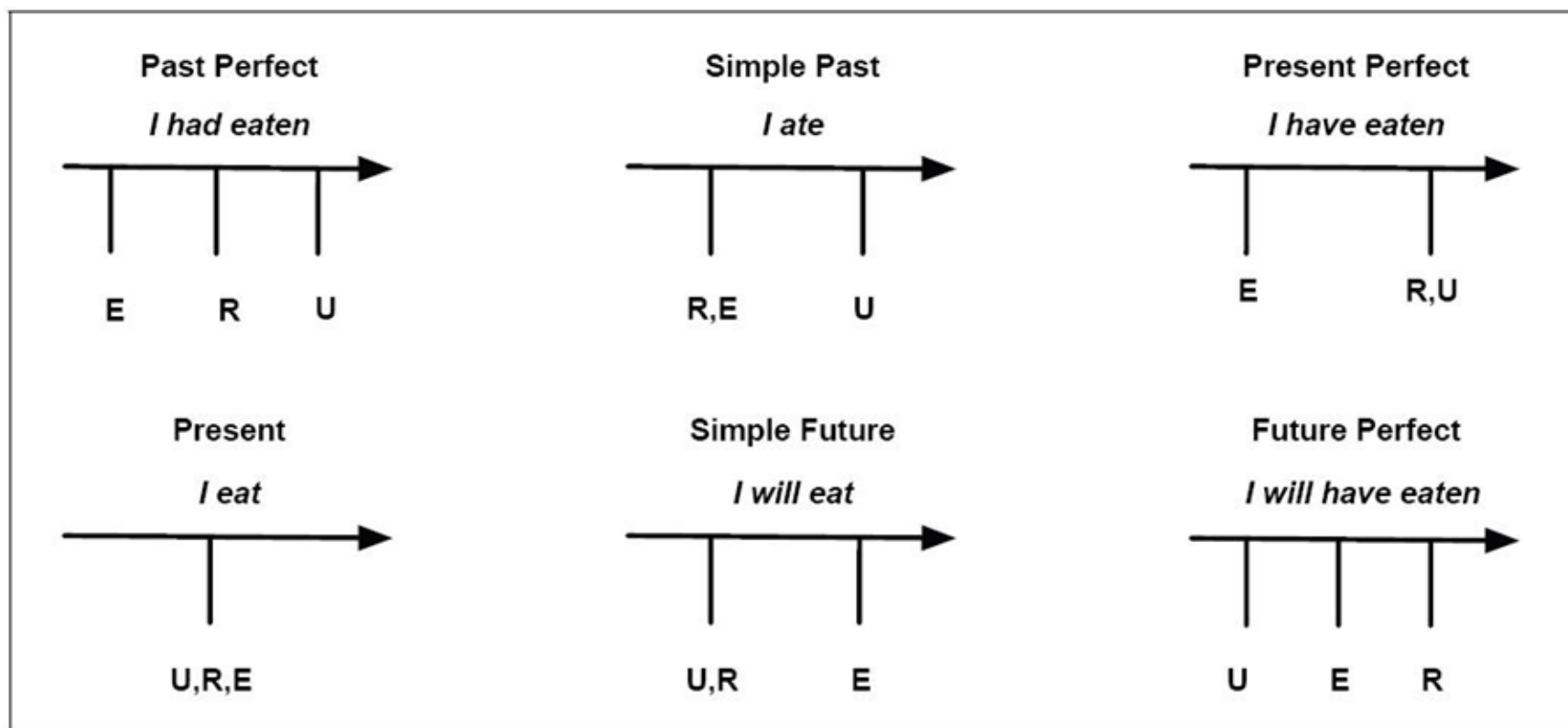


... I reject the contention that an important theoretical difference exists between formal and natural languages ...

English as formal language, pp. 188-221, Linguaggi nella Società e nella Tecnica. Convegno promosso dalla Ing. C. Olivetti & C. S. p. A. per il centenario della nascita di Camillo Olivetti, Museo Nazionale della Scienza e della Tecnica, Milano, 14-17 Ottobre 1968 (Ristampato, Edizioni di Comunità, 1970)

Stiamo considerando solo parte del significato!

- Struttura Predicato-Argomento
- Il tempo? -> Reference time: Reichenbach, 1947

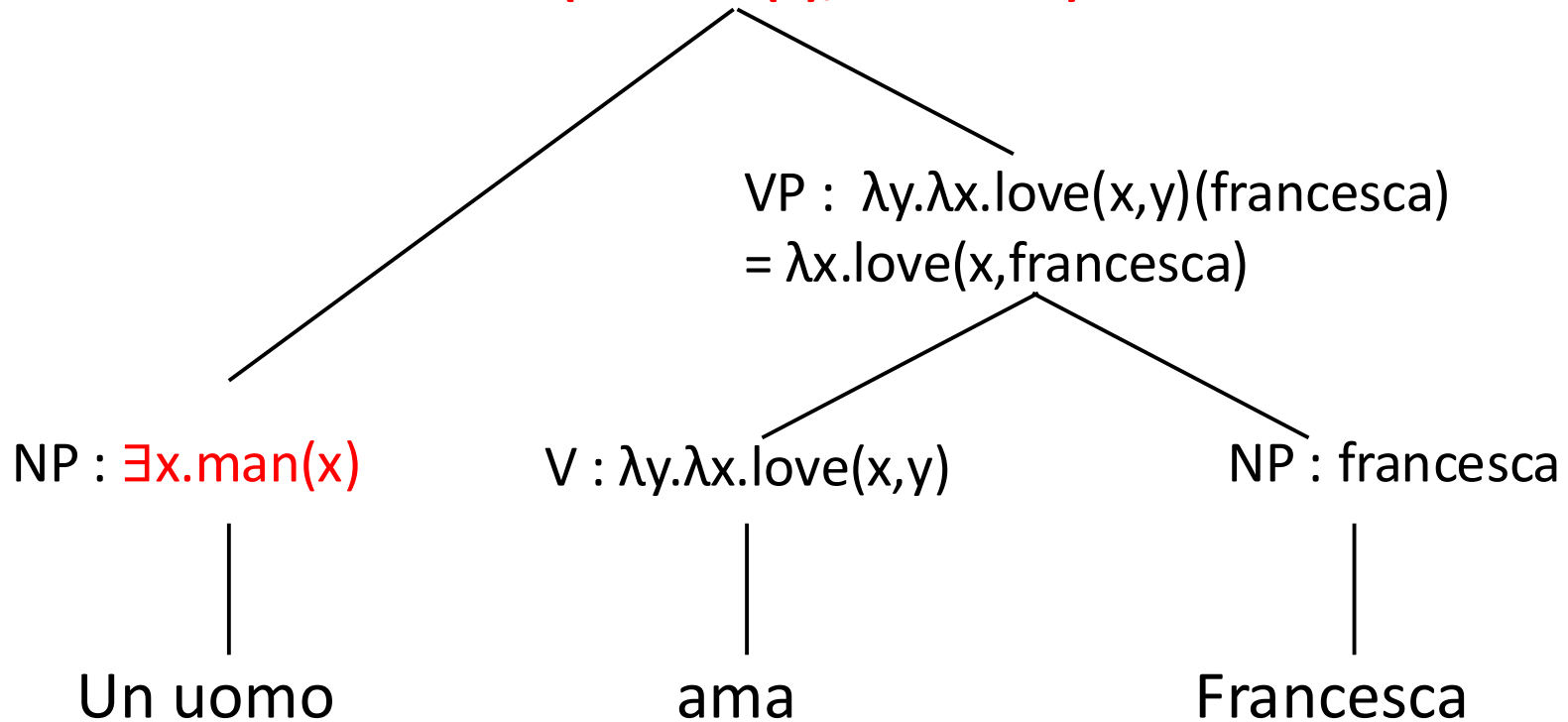


Outline

- SHRDLU & Chat-80
- Computational Semantics fundamentals
- λ abstraction
- **CS per:**
 - >Articoli indeterminativi ->Nomi propri ->Verbi transitivi
 - ~~->Cordinazione dei nomi~~ ~~->Quantificatori universali~~
- NLTK

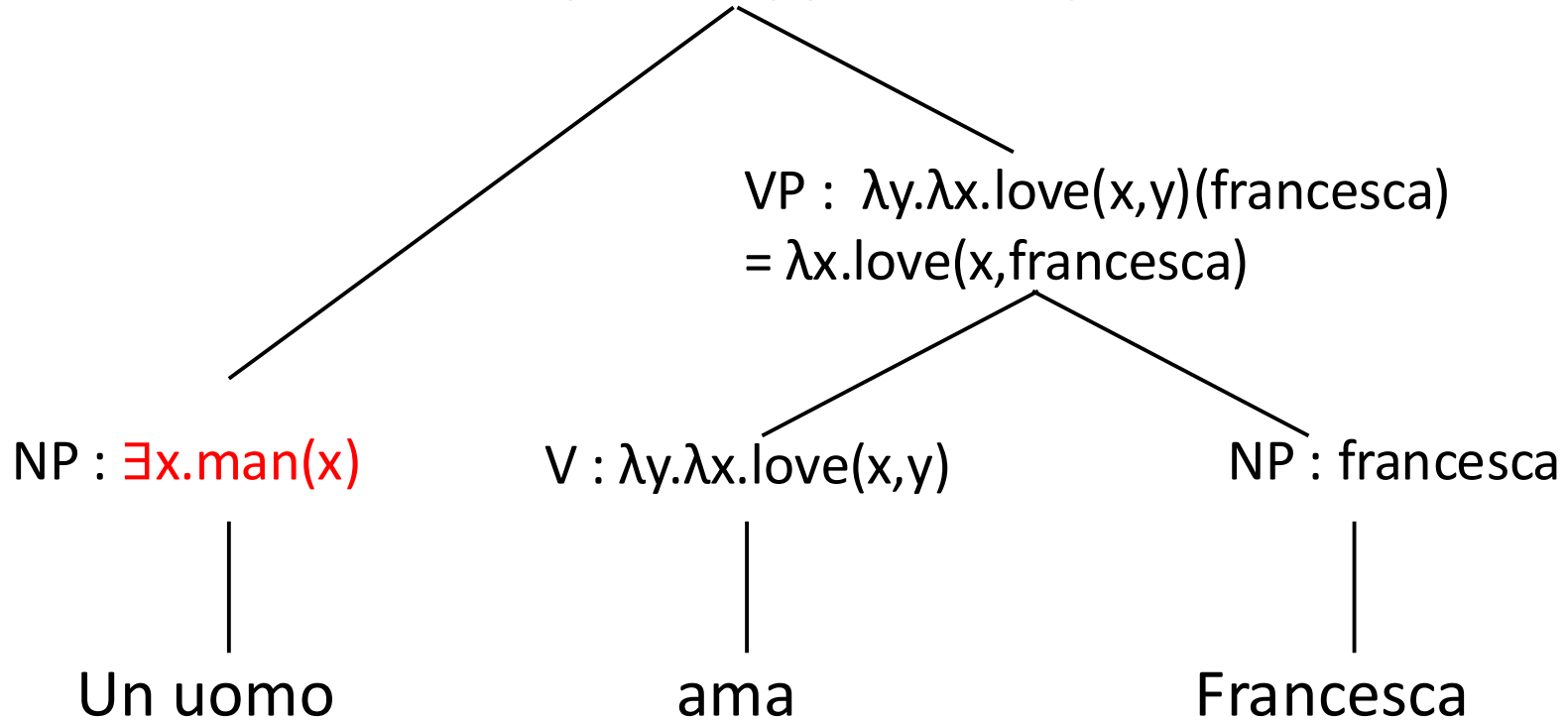
Esempio 2: gli articoli

$S : \lambda x.\text{love}(x,\text{francesca})(\exists x.\text{man}(x))$
 $= \text{love}(\exists x.\text{man}(x),\text{francesca})$



Esempio 2: gli articoli

$S : \lambda x.\text{love}(x,\text{francesca})(\exists x.\text{man}(x))$
 $= \text{love}(\exists x.\text{man}(x),\text{francesca})$



$\exists x.\text{man}(x)$ significa “*c'è un uomo ...*” e non “*un uomo*”

Come trattare gli articoli -> reverse engineering

L'uomo saggio inizia dalla fine e finisce col principio (cit. ?)

Vorremmo:

Un uomo ama Francesca =>

$\exists z (\text{man}(z) \wedge \text{love}(z, \text{francesca}))$

$\exists z (\lambda y. \text{man}(y)(z) \wedge \lambda x. \text{love}(x, \text{francesca})(z))$

Dobbiamo permettere l'astrazione anche sui predicati:

$(\lambda Q. \exists z (\lambda y. \text{man}(y)(z) \wedge Q(z))) (\lambda x. \text{love}(x, \text{francesca}))$

$(\lambda P. \lambda Q. \exists z (P(z) \wedge Q(z))) (\lambda x. \text{love}(x, \text{francesca})) (\lambda y. \text{man}(y))$

Come trattare gli articoli

Nel lessico:

$$DT : \lambda P. \lambda Q. \exists z (P(z) \wedge Q(z)) \rightarrow un$$

Come trattare gli articoli

Nel lessico:

$$DT : \lambda P. \lambda Q. \exists z (P(z) \wedge Q(z)) \rightarrow \text{un}$$
$$DT : \lambda P. \lambda Q. \forall z (P(z) \rightarrow Q(z)) \rightarrow \text{tutti}$$
$$DT : \lambda P. \lambda Q. \forall z (P(z) \rightarrow \neg Q(z)) \rightarrow \text{nessun}$$

Una semplicissima grammatica semantica (2)

Lessico

DT : $\lambda P.\lambda Q.\exists z (P(z) \wedge Q(z)) \rightarrow$ un

N : $\lambda y.\text{man}(y) \rightarrow$ uomo

NP : francesca $\rightarrow$ Francesca

V : $\lambda y.\lambda x.\text{love}(x, y) \rightarrow$ ama

Regole di composizione

VP : $f(a) \rightarrow$ V : f NP : a

S : $f(a) \rightarrow$ NP : f VP : a

Esempio 2: gli articoli

$S : (\lambda Q. \exists z (\text{man}(z) \wedge Q(z))) (\lambda x. \text{loves}(x, \text{francesca}))$
 $= \exists z (\text{man}(z) \wedge (\lambda x. \text{loves}(x, \text{francesca}))(z))$
 $= \exists z (\text{man}(z) \wedge \text{loves}(z, \text{francesca}))$

$NP : (\lambda P. \lambda Q. \exists z (P(z) \wedge Q(z))) (\lambda y. \text{man}(y))$
 $= \lambda Q. \exists z ((\lambda y. \text{man}(y))(z) \wedge Q(z))$
 $= \lambda Q. \exists z (\text{man}(z) \wedge Q(z))$

$VP : \lambda y. \lambda x. \text{love}(x, y)(\text{francesca})$
 $= \lambda x. \text{love}(x, \text{francesca})$

$DT : \lambda P. \lambda Q. \exists z (P(z) \wedge Q(z))$

$N : \lambda y. \text{man}(y)$

$V : \lambda y. \lambda x. \text{love}(x, y)$

$NP : \text{francesca}$

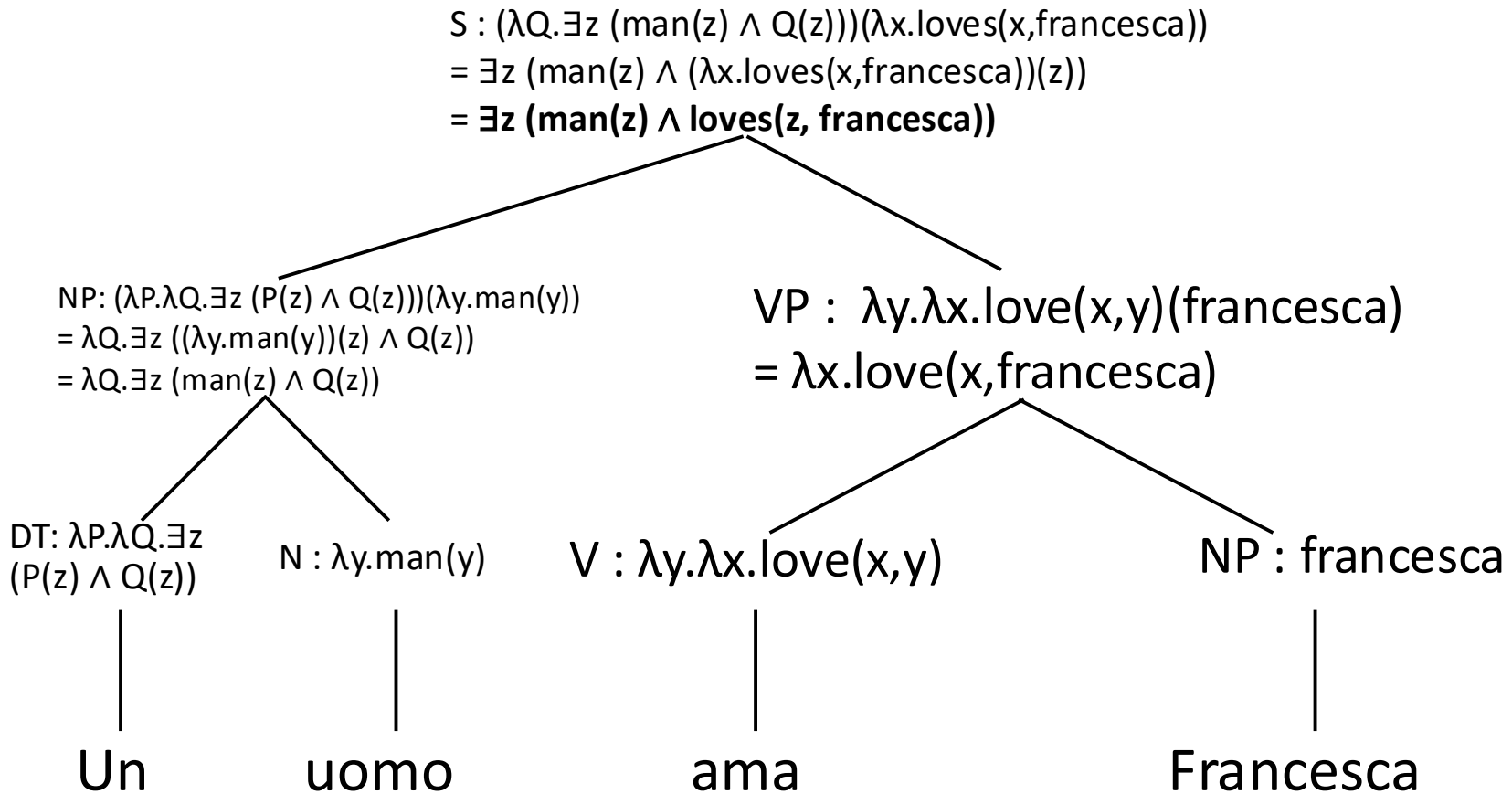
Un

uomo

ama

Francesca

Esempio 2: gli articoli



“Paolo ama Francesca” funziona **ancora?**

$\lambda x. \text{love}(x, \text{francesca}) @ (\text{paolo})$ **VS.** $(\text{paolo}) @ (\lambda x. \text{love}(x, \text{francesca}))$

Outline

- SHRDLU & Chat-80
- Computational Semantics fundamentals
- λ abstraction
- CS per:
 - >Articoli indeterminativi ->**Nomi propri** ->Verbi transitivi
 - ~~->Cordinazione dei nomi~~ ~~->Quantificatori universali~~
- NLTK

Problema con i nomi propri soggetto

$S : f(a) \rightarrow NP : f \quad VP : a$

$(\text{paolo}) @ (\lambda x. \text{love}(x, \text{francesca}))$

Problema con i nomi propri soggetto

~~S : f(a) → NP : f VP : a~~

~~(paolo) @ (λx.love(x, francesca))~~

λP.P(paolo))

λP.P(paolo) @ (λx.love(x, francesca)) →

λx.love(x, francesca) @ (paolo) →

love(paolo, francesca))

Type-raising

old type: e

new type: (e → t) → t

Una semplicissima grammatica semantica (3)

Lessico

DT : $\lambda P. \lambda Q. \exists z (P(z) \wedge Q(z)) \rightarrow$ un

N : $\lambda y. \text{man}(y) \rightarrow$ uomo

NP : $\lambda P. P(\text{paolo}) \rightarrow$ Paolo

NP : $\lambda P. P(\text{francesca}) \rightarrow$ Francesca

V : $\lambda y. \lambda x. \text{love}(x, y) \rightarrow$ ama

Regole di composizione

VP : $f(a) \rightarrow$ V : f NP : a

S : $f(a) \rightarrow$ NP : f VP : a

Outline

- SHRDLU & Chat-80
- Computational Semantics fundamentals
- λ abstraction
- CS per:
 - >Articoli indeterminativi ->Nomi propri ->**Verbi transitivi**
 - ~~->Cordinazione dei nomi~~ ~~->Quantificatori universali~~
- NLTK

Problema con i verbi transitivi

“ama Francesca” funziona **ancora**?

Per i verbi transitivi:

$\text{ama} \leftarrow V : \lambda y. \lambda x. \text{love}(x, y)$

Ma con i nomi propri:

$\text{Francesca} \leftarrow \text{NP}:$

$\lambda P. P(\text{francesca})$

quindi “ama Francesca”:

$\lambda y. \lambda x. \text{love}(x, y) @ \lambda P. P(\text{francesca}) \rightarrow (\lambda x. \text{love}(x, \lambda P. P(\text{francesca})))$



Problema con i verbi transitivi

“ama Francesca” funziona **ancora**?

Per i verbi transitivi:

$\text{ama} \leftarrow V : \lambda y. \lambda x. \text{love}(x, y)$

Ma con i nomi propri:

$\text{Francesca} \leftarrow \text{NP}:$

$\lambda P. P(\text{francesca})$

quindi “ama Francesca”:

$\lambda y. \lambda x. \text{love}(x, y) @ \lambda P. P(\text{francesca}) \rightarrow (\lambda x. \text{love}(x, \lambda P. P(\text{francesca})))$

Type-raising

old type: $e \rightarrow (e \rightarrow t)$

new type: $((e \rightarrow t) \rightarrow t) \rightarrow (e \rightarrow t)$

Soluzione: ancora type raising!

$\text{ama} \leftarrow V: \lambda R. \lambda x. R(\lambda y. \text{love}(x, y))$

Paolo ama Francesca

$S : \lambda P.P(\text{paolo})\lambda x.\text{love}(x,\text{francesca})$
 $= \lambda x.\text{love}(x,\text{francesca})(\text{paolo})$
 $= \text{love}(\text{paolo},\text{francesca})$

$VP: \lambda R.\lambda x.R(\lambda y.\text{love}(x,y))(\lambda Q.Q(\text{francesca}))$
 $= \lambda x.(\lambda Q.Q(\text{francesca}))(\lambda y.\text{love}(x,y))$
 $= \lambda x.(\lambda y.\text{love}(x,y))(\text{francesca})$
 $= \lambda x.\text{love}(x,\text{francesca})$

$NP : \lambda P.P(\text{paolo})$

$V : \lambda R.\lambda x.R(\lambda y.\text{love}(x,y))$

$NP : \lambda Q.Q(\text{francesca})$

Paolo

ama

Francesca

Una semplicissima grammatica semantica (4)

nomi comuni	<i>uomo</i>	$\lambda x.\text{man}(x)$
nomi propri	<i>Paolo</i>	$\lambda P.P(\text{Paolo})$
verbi intr.	<i>corre</i>	$\lambda x.\text{run}(x)$
verbi tr.	<i>ama</i>	$\lambda R.\lambda x.R(\lambda y.\text{love}(x, y))$
articoli	<i>un</i>	$\lambda P.\lambda Q.\exists z(P(z) \wedge Q(z))$

Punti chiave

- extra λ per gli NP
- astrazione sui predicati
- inversione di controllo: NP_subj come funzione e VP come argomento

Sistematicità

Outline

- SHRDLU & Chat-80
- Computational Semantics fundamentals
- λ abstraction
- CS per:
 - >Articoli indeterminativi ->Nomi propri ->Verbi transitivi
 - >~~Cordinazione dei nomi~~ ->~~Quantificatori universali~~
- NLTK

Coordinazione dei nomi

- *Paolo e Francesca corrono*
 - $\text{run}(\text{Paolo}) \wedge \text{run}(\text{Francesca})$

Coordinazione dei nomi

- *Paolo e Francesca corrono*
 - $\text{run}(\text{Paolo}) \wedge \text{run}(\text{Francesca})$
- Aggiungiamo al lessico:
 - $\text{CC} : \lambda X. \lambda Y. \lambda R. (X(R) \wedge Y(R)) \rightarrow e$

Paolo e Francesca corrono

$\lambda R. (R(\text{paolo}) \wedge (R(\text{francesca})))(\lambda x. \text{run}(x))$
 $= \text{run}(\text{paolo}) \wedge \text{run}(\text{francesca})$

$(\lambda Y. \lambda R. (R(\text{paolo}) \wedge Y(R)))(\lambda P. P(\text{francesca}))$
 $= \lambda R. (R(\text{paolo}) \wedge (\lambda P. P(\text{francesca}))(R))$
 $= \lambda R. (R(\text{paolo}) \wedge (R(\text{francesca})))$

$(\lambda X. \lambda Y. \lambda R. (X(R) \wedge Y(R)))(\lambda P. P(\text{paolo}))$
 $= (\lambda Y. \lambda R. (\lambda P. P(\text{paolo}))(R) \wedge Y(R))$
 $= (\lambda Y. \lambda R. (R(\text{paolo}) \wedge Y(R)))$

$\lambda P. P(\text{paolo})$

Paolo

$\lambda X. \lambda Y. \lambda R. (X(R) \wedge Y(R))$

e

$\lambda P. P(\text{francesca})$

Francesca

$\lambda x. \text{run}(x)$

corrono

E per le altre coordinazioni?

- Paolo corre e vince
- Paolo mangia pane e cipolla
- Paolo corre e Francesca vede la gara

Outline

- SHRDLU & Chat-80
- Computational Semantics fundamentals
- λ abstraction
- CS per:
 - >Articoli indeterminativi ->Nomi propri ->Verbi transitivi
 - >~~Cordinazione dei nomi~~ ->**Quantificatori universali**
- NLTK

Scope ambiguity

In this country, a woman gives birth every 15 minutes.

Scope ambiguity

In this country, a woman gives birth every 15 minutes.

Our job is to find that woman and stop her.

Groucho Marx

Scope ambiguity

Every man loves a woman

Lettura 1:

$$\forall x (\text{man}(x) \rightarrow \exists y (\text{woman}(y) \wedge \text{love}(x, y)))$$

Lettura 2:

$$\exists y (\text{woman}(y) \forall x (\text{man}(x) \rightarrow \text{love}(x, y)))$$

$$\lambda Q \lambda P (\forall x (Q(x) \rightarrow P(x))) \rightarrow \text{Every}$$

Avverbi: *Paolo ama Francesca dolcemente*

- **sweetly(love(P,F))** : second order!!!!
- **love(P,F,sweetly)**: e se ho più avverbi??

Avverbi: *Paolo ama Francesca dolcemente*

- Soluzione: **reificare l'evento**, ovvero definire una variabile che identifica l'evento (Davidson)

$$\exists e \text{ love}(e, P, F) \wedge \text{sweetly}(e)$$

- Si può generalizzare anche per gli argomenti:
 - neo-Davidsonian style (Parson) ->

$$\exists e \text{ love}(e) \wedge \text{agent}(e, P) \wedge \text{patient}(e, F)$$

Paolo ama Francesca dolcemente

ESERCIZIO: costruire la derivazione (albero e lambda-FoL termini) in stile Neo-Davidsoniano per:

Paolo ama Francesca dolcemente



$\exists e \text{ love}(e) \wedge \text{agent}(e,P) \wedge \text{patient}(e,F) \wedge \text{sweetly}(e)$

(Groningen & Parallel) Meaning Bank

- Annotazione completa
 - Sintassi: CCG
 - Semantica: Discourse Representation Theory (DRT), a formal theory of meaning developed by the philosopher of language Hans Kamp (Kamp, 1981; Kamp and Reyle, 1993)
-> Equivalente a FoL.
- GMB: <http://gmb.let.rug.nl>
 - *A free semantically annotated corpus that anyone can edit!*
- PMB: <http://pmb.let.rug.nl>

Outline

- SHRDLU & Chat-80
- Computational Semantics fundamentals
- λ abstraction
- CS per:
 - >Articoli indeterminativi ->Nomi propri ->Verbi transitivi
 - ~~->Cordinazione dei nomi ->Quantificatori universali~~
- NLTK

nltk.sem [Garrette & Klein 2008]

nltk.sem package (Python):

- **First-order logic & lambda calculus**
- Theorem proving, model building, & model checking
- DRT & DRSs
- Cooper storage, hole semantics, glue semantics
- Linear logic

<http://www.nltk.org/book/ch10.html>

Paolo ama Francesca in NLTK

```
>>> from nltk import load_parser
>>> parser = load_parser('./simple-sem-it.fcfg', trace=0)
>>> sentence = 'Paolo ama Francesca'
>>> tokens = sentence.split()
>>> for tree in parser.parse(tokens):
...     print(tree.label()['SEM'])
...
love(paolo, francesca)
```

simple-sem-it.fcfg

```
## Natural Language Toolkit: A simple example for Italian
## Author: Alessandro Mazzei <mazzei@di.unito.it>
## To see more complex example -> simple-sem.fcfg

% start S
#####
# Grammar Rules
#####

S[SEM = <?subj(?vp)>] -> NP[SEM=?subj] VP[SEM=?vp]
NP[SEM=?np] -> PropN[SEM=?np]
VP[SEM=<?v(?obj)>] -> V[SEM=?v] NP[SEM=?obj]
#####
# Lexical Rules
#####

PropN[SEM=<\P.P(paolo)>] -> 'Paolo'
PropN[SEM=<\P.P(francesca)>] -> 'Francesca'
V[SEM=<\X x.X(\y.love(x,y))>] -> 'ama'
```

Un esempio più complesso in Inglese

```
>>> from nltk import load_parser
>>> parser = load_parser('./simple-sem.fcfg', trace=0)
>>> sentence = 'Angus gives a bone to every dog'
>>> tokens = sentence.split()
>>> for tree in parser.parse(tokens):
...     print(tree.label() ['SEM'])
...
all z2.(dog(z2) -> exists z1.(bone(z1) & give(angus,z1,z2)))
```

<http://www.nltk.org/howto/inference.html>

EACL 2024 invited speaker: Prompting is *not* all you need! Or why Structure and Representations still matter in NLP

Abstract: Recent years have witnessed the rise of increasingly larger and more sophisticated language models (LMs) capable of performing every task imaginable, sometimes at (super)human level. In this talk, I will argue **that there is still space for specialist models in today's NLP landscape**. Such models can be dramatically more efficient, inclusive, and explainable. I will focus on two examples, opinion summarization and crosslingual semantic parsing and show how these two seemingly unrelated tasks can be addressed by **explicitly learning task-specific representations**. I will show how **such representations can be further structured** to allow search and retrieval, evidence-based generation, and cross-lingual alignment. Finally, I will discuss **why we need to use LLMs for what they are good at and remove the need for them to do things that can be done much better by smaller models**.

20° March 2024: <https://2024.eacl.org/program/invited/>

<https://underline.io/lecture/95982-keynote-mirella-lapata>